

1 自己编写一个分支定界法程序，然后求解如下

设某服装加工厂有 5 个生产车间，可以用 6 种不同的成品布料（单位为 m）加工成不同的服装销售。对于第 i 个生产车间分别利用第 j 种布料进行加工生产后，可以获得利润为 r_{ij} (元/m) ($i = 1, 2, \dots, 5; j = 1, 2, \dots, 6$)，第 j 种布料的价格为 a_j (元/m) ($j = 1, 2, \dots, 6$)，具体的数据如表 4-2 所示。

该工厂现有资金 40 万元，为了充分利用这些有限的资金，根据各车间的实际生产需求，工厂要求每个车间每种布料至少加工 1000 m，每个车间的总加工能力最多 10000 m 那么试问该工厂每种布料应购买多少米，又如何分配给所属的 5 个车间，使得总利润最大？

表4-2 布料单价及加工利润						
布料 利润/元 车间	1	2	3	4	5	6
车间一	4	3	4	4	5	6
车间二	3	4	5	3	4	5
车间三	5	3	4	5	5	4
车间四	3	3	4	4	6	6
车间五	3	3	3	4	5	7
布料单价/(元/米)	6	6	7	8	9	10

决策变量为第 i 个车间价格第 j 种布料 x_{ij}

目标函数为 $\max \sum_{i=1}^5 \sum_{j=1}^6 x_{ij} r_{ij}$

约束条件有：

- 加工能力的约束 $\sum_{j=1}^6 x_{ij} \leq 10000, i = 1, 2, 3, 4, 5$
- 加工要求的约束 $x_{ij} \geq 1000, i = 1, 2, 3, 4, 5, j = 1, 2, 3, 4, 5, 6$
- 资金约束 $\sum_{j=1}^6 a_j \sum_{i=1}^5 x_{ij} \leq 400000$
- 非负整数约束 $x_{ij} \geq 0$ 且 x_{ij} 为整数

得到基本条件后我们分析分支定界法的原理：

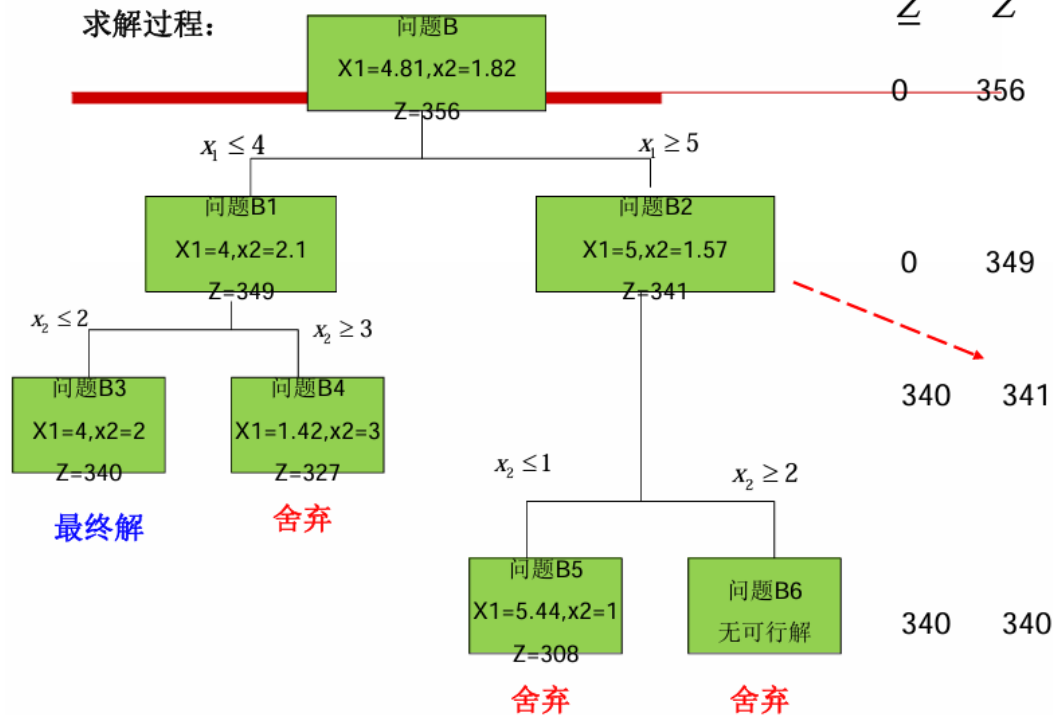
1. 先对原始问题进行线性规划求解
2. 对求出的解中的非整数决策变量重新定义上下界进行分支
3. 对每个新问题求解，同时更新最优值的上下限
4. 直到得到整数解且上下限一致，即收敛到一个值

但是这样我们会遇到一些问题：

1. matlab中自带的linprog函数求的是最小值问题，这样最优值上下界的更新要反过来。
2. 另外，更新上下界是为了人为计算时简化步骤，可以舍去一些不必要的区域；但是对于计算机来说，算力足够，不必要简化步骤，可以直接遍历所有的区域，即第一次分支后，对所有的非整数决策变量都进行不断的分支，直到最后分得的若干区域全部都是整数解，而且在遍历时，将每个有整数解的区域对应的最优值记录下来，与初始最优值对比并且更新，这样也可以保证得到的解一定为最优解，因为如果这个区域对应的线性规划解是整数，那么其整数规划解就是线性规划解。



求解过程:



参考ppt里的二叉树，根节点是初始问题，然后根据不同的决策变量分支一次次进行二分，最后得到的不可再分的若干节点便是解为整数解的区域。

1.2 代码实现

首先定义目标函数，约束条件，决策变量

```
%利润矩阵
r=[4 3 4 4 5 6;
   3 4 5 3 4 5;
   5 3 4 5 5 4;
   3 3 4 4 6 6;
   3 3 3 4 5 7];

%布料单价
a=[6 6 7 8 9 10];

num_shop=5;%车间数
num_cloth=6;%布的种类
%目标函数
r=r';
c=-r(:);%取负数转化为最小化问题
num_vars=num_shop*num_cloth;%决策变量数
%加工能力约束，一个车间不超过10000
A=zeros(num_shop,num_vars);
b=zeros(num_shop,1);
for i = 1:num_shop
    A(i,(i-1)*num_material+1:i*num_material)=1;
    b(i)=10000;
end
%资金能力约束
a_expanded=zeros(1, num_vars);
for i=1:num_shop
    a_expanded((i-1)*num_material+1:i*num_material)=a;
```



```

end
%检查解是否为整数解
[is_integer,fractional_idx]=check_integer(x);
if is_integer
    %是整数解，更新最优解
    if -obj>best_obj
        best_obj=-obj;
        best_x=x;
    end
else
    %不是整数解，进行分支
    branch_var=fractional_idx(1); %选择第一个非整数变量进行分支

    %创建左子节点（添加上限约束）
    left_node=current_node;
    left_node.ub(branch_var)=floor(x(branch_var));
    left_node.is_integer=false;
    left_node.obj=inf;

    %创建右子节点（添加下限约束）
    right_node = current_node;
    right_node.lb(branch_var)=ceil(x(branch_var));
    right_node.is_integer=false;
    right_node.obj=inf;

    %将子节点加入队列
    node_queue=[node_queue, {left_node}, {right_node}];
end
end

```

可见该队列对二叉树的遍历是层序遍历

结果展示

```

x_matrix=[];
for i=1:5
    x_matrix=[x_matrix,best_x(6*i-5:6*i)];
end
x_matrix=x_matrix';
fprintf('每个车间每种布料的分配量:\n');
disp(x_matrix);
fprintf('最大总利润:%.2f元\n',best_profit);
fprintf('遍历的节点数:%d\n',nodes_explored);
%验证结果
sum=0;
r=r';
for i=1:5
    for j=1:6
        sum=sum+x_matrix(i,j)*r(i,j);
    end
end
disp(sum);

```

最终结果为

每个车间每种布料的分配量:

1000	1000	1000	1000	1000	5000
1000	1000	5000	1000	1000	1000
4333	1000	1000	1001	1666	1000
1000	1000	1000	1000	5000	1000
1000	1000	1000	1000	1000	5000

最大总利润:243000.00元

遍历的节点数:3

243000

遍历过程如下

1.0e+03 *

1.0000	1.0000	1.0000	1.0000	1.0000	5.0000
1.0000	1.0000	5.0000	1.0000	1.0000	1.0000
4.3333	1.0000	1.0000	1.0000	1.6667	1.0000
1.0000	1.0000	1.0000	1.0000	5.0000	1.0000
1.0000	1.0000	1.0000	1.0000	1.0000	5.0000

找到最优解。

1000	1000	1000	1000	1000	5000
1000	1000	5000	1000	1000	1000
4333	1000	1000	1001	1666	1000
1000	1000	1000	1000	5000	1000
1000	1000	1000	1000	1000	5000

找到最优解。

1.0e+03 *

1.0000	1.0000	1.0000	1.0000	1.0000	5.0000
1.0000	1.0000	4.9993	1.0000	1.0000	1.0007
4.3340	1.0000	1.0000	1.0000	1.6660	1.0000
1.0000	1.0000	1.0000	1.0000	5.0000	1.0000
1.0000	1.0000	1.0000	1.0000	1.0000	5.0000

遍历了三个节点

遍历了三个节点

但是当我们不定义上下界，直接对原c, A, b进行线性求解

```
[x,fv]=linprog(c,A,b,[],[]);  
x_matrix=[];  
for i=1:5  
    x_matrix=[x_matrix,x(6*i-5:6*i)];  
end  
x_matrix=x_matrix';  
disp(x_matrix);  
disp(-fv);
```

找到最优解。

1000	1000	1000	1000	1000	5000
1000	1000	5000	1000	1000	1000
5000	1000	1000	1000	1000	1000
1000	1000	1000	1000	5000	1000
1000	1000	1000	1000	1000	5000

243000

得到的结果如下，最终的目标函数值相同，但是决策变量的分配不同，原因在于调用linprog函数时是否传入上下界lb、ub参数，**但是理论上无论是否传入，其约束条件已经隐含了上下界条件**，具体原因可能和linprog函数的具体算法有关。

1.3 最终结果

用分支定界法的结果为

每个车间每种布料的分配量：

1000	1000	1000	1000	1000	5000
1000	1000	5000	1000	1000	1000
4333	1000	1000	1001	1666	1000
1000	1000	1000	1000	5000	1000
1000	1000	1000	1000	1000	5000

最大总利润：243000.00元

遍历的节点数：3

243000

直接线性规划求解的结果为

找到最优解。

1000	1000	1000	1000	1000	5000
1000	1000	5000	1000	1000	1000
5000	1000	1000	1000	1000	1000
1000	1000	1000	1000	5000	1000
1000	1000	1000	1000	1000	5000

243000

两者从目标函数的角度上都是最优解。

再从成本的角度去看，前者的成本恰好为400000，后者的成本为398000。所以从现实角度后者是最优解，从数学模型角度两者都是最优解。

2 教材练习4.1

一个旅行者要在背包里装一些最有用的东西，但限制最多只能带 bkg 物品，每件物品只能是整件携带，对每件物品都规定了一定的“使用价值”(有用的程度)，如果共有n件物品，第i件物品重 a_j kg，其价值为 c_j ($j=1,2,\dots,n$)，问题是:在携带的物品总重量不超过 bkg 的条件下，携带哪些物品可使总价值最大？

首先列写模型，然后自己赋值进行程序求解

自行赋值，假设最大总重量b=25kg

物品	1	2	3	4	5	6	7	8	9	10
重量	3	5	7	4	2	1	9	5	4	2

物品	1	2	3	4	5	6	7	8	9	10
价值	4	6	7	3	1	1	10	6	3	3

决策变量 $x_i=1$ (拿第 i 个物品) or 0 (不拿第 n 个物品)

目标函数 $\max \sum_{i=1}^{10} x_i c_i$

约束条件 $\sum_{i=1}^{10} x_i b_i \leq 30$

2.1 代码实现与结果

本题是个0-1问题，一般用枚举法，对于本题，共有 2^{10} 种组合，如果用显枚举法，每次循环都要求总价值和总重量，虽然对于计算机来说计算量不大，但是时间还可以减少。**用隐枚举法，用一个不断更新的下界作为过滤条件**，对于一个新值，先求其总价值，如果大于这个新值，就再计算总重量是否满足约束条件，如果满足，更新best_x值与下界。这样不仅可以避免一大部分对于总重量的计算，也可以实时更新最优解与最优值。

而各种组合，实际上可以看成是0-1023各个十进制数的二进制，从0000000000到1111111111。由于遍历时二进制长度不一样，用dec2bin(i-1, 10)统一规定为10位二进制数。

计算时遇到了一个诡异的现象，将十进制用dec2bin函数转化为二进制时，并不是转化为一个只有1与0的数组，比如a=dec2bin(5)，输出101，用a(1)*5，输出的不是 $1*5=5$ ，而是245；用a(2)*5，输出的不是0，而是240。所以说用其运算时，1实际上对应49，0实际上对应48，因此计算时要先减去48。

```

a=[3,5,7,4,2,1,9,5,4,2];%重量
c=[4,6,7,3,1,1,10,6,3,3];%价值
b=25;%最大重量
best_x=[];% 最优解
best_value=0;%最优值
best_weight=0;%最优值对应的总重量
lb=0;%隐枚举的下界
for i=1:1024%共有2^10种可能
    tool=dec2bin(i-1,10);%用二进制表示组合
    sum_weight=0;%总重量
    sum_value=0;%总价值
    for j=1:10%先求总价值
        sum_value=sum_value+(tool(j)-48)*c(j);
    end
    if sum_value>lb%看是否满足隐枚举条件
        for j=1:10
            sum_weight=sum_weight+(tool(j)-48)*a(j);
        end
        if sum_weight<=b%再判断是否满足重量约束
            lb=sum_value;%更新下界
            best_x=tool;
            best_value=sum_value;
            best_weight=sum_weight;
        end
    end
end
disp(best_x);
disp(best_value);
disp(best_weight);

```


输出为

1100011101

30

25

即选第1、2、6、7、8、10件物品时是最优解，此时总重量25kg，总价值30